

UNIVERSITY OF NOTTINGHAM

School of Computer Sciences

COMP1234 - (2023-24)

Assignment 1- Exam Timetabling

Symbolic Artificial Intelligence (COMP3008)

Submitted by:
Nderon Beqiri

Contents

Approaching The Problem 3
Creating The Functions 4
Modelling Student Taking Exam 4
Defining Room Capacities 4
Determining Exam Capacities 4
Applying The Rules 5
Testing 6
Creating Own Examples 6
How will we test? 6
Testing All Instances 8
add_exam() Function 9
Printing The Timetable 10
Final Test 11
Final Tweaks 12
Conclusion 12
Additional Features 12

Approaching The Problem

To make things simpler, we start by creating variables, each taking on the value of the instance. This simplifies the code and avoids constant references to 'instance.', in order to improve the quality of the code.

```
#Create variables and assign values of instance
no_of_exams = instance.number_of_exams
no_of_students = instance.number_of_students
no_of_slots = instance.number_of_slots
no_of_rooms = instance.number_of_rooms
room_cap = instance.room_capacities
exams_to_students = instance.exams_to_students
```

Figure 1: Assigning values based on 'instance.'

The project briefing states the sets we are given; Exams, Students, Rooms, and Time slots. Using this information, we therefore can create datatypes for each, all of which store as much data as given to us by the instance (no_of_whatIsNecessary). This results in four Z3 datatypes, with differing individual components; exams, students, rooms and time slots.

```
#Create datatypes necessary E S R T
E = z3.Datatype('E')
for i in range (no_of_exams):
  E.declare(f'{i}')
E = E.create()

S = z3.Datatype('S')
for i in range (no_of_students):
  S.declare(f'{i}')
S = S.create()

R = z3.Datatype('R')
for i in range (no_of_rooms):
  R.declare(f'{i}')
R = R.create()

T = z3.Datatype('T')
for i in range (no_of_slots):
  T.declare(f'{i}')
T = T.create()
```

Figure 2: Creating datatypes for each set

Creating The Functions

Modelling Student Taking Exam

The function ‘take_exam()’ is created to model whether a student, S, is taking an exam, E, in a specific room, R, and at a specific time slot, T. The function returns a Boolean value indicating whether the student is taking the specified exam in the given room and time slot.

Defining Room Capacities

The function ‘room_capacity()’ is created to represent the capacity of a room. The capacities for each room are stored in list ‘room_cap’. The solver checks the capacity using the ‘room_capacity()’ function based on what's in the ‘room_cap’ list."

Determining Exam Capacities

The function ‘exam_capacity()’ is created to represent the capacity of an exam. We create a list called ‘exam_cap’ and initialise it with zeros, and its length is determined by ‘no_of_exams’. This list keeps track of how many students are taking each exam.
We go through a set of pairs, where each pair represents an (exam, student) relationship. For every pair, the count of students for that exam is increased by 1 in the ‘exam_cap’ list. It must be the case that for all (exam, student) pair, there exists a room and a time slot. The solver checks the capacity using the ‘exam_capacity()’ function based on what's in the ‘exam_cap’ list."

```
#Create functions
# 1) Student taking exam
take_exam = z3.Function('take_exam',E, S, R, T, z3.BoolSort())

# 2) Room capacity
room_capacity = z3.Function('room_capacity',R, z3.IntSort())
# Store room_capacities[] list
for i in range (len(room_cap)):
  solver.add(room_capacity(R.constructor(i)())==room_cap[i])

# 3) Exam capacity
exam_capacity = z3.Function('exam_capacity',E, z3.IntSort())
# Create array to count capacity for each exam
exam_cap = [0] * no_of_exams
# Update the count for each exam based on the (exam, student) pairs
for i, j in exams_to_students:
  exam_cap[i] += 1
  # Ensure there exists a room and time slot for each (exam, student) pair
  solver.add(z3.ForAll([s1,e1],z3.Exists([r1, ts1], take_exam(E.constructor(i)(), S.constructor(j)(), r1, ts1))))
# Store exam_cap[] list
for i in range (no_of_exams):
  solver.add(exam_capacity(E.constructor(i)())==exam_cap[i])
```

Figure 3: Functions

Applying The Rules

Below are my interpretations of each rule and a simple explanation depicting their functionality.

1) ∀ts1, ts2, r1, r2, s1, s2,
   (e1!=e2 ∧ take_exam(e1,s1,r1,ts1) ∧ take_exam(e2,s2,r2,ts2))→
   (r1 != r2 V ts1 != ts2)

Rule 1 states that for all students, if two exams are different, then either the room or time slots must be different.

2) ∀ts1, ts2, r1, r2, s1, s2,
   (take_exam(e1,s1,r1,ts1) ∧ take_exam(e2,s2,r2,ts2) ∧ ts1==ts2 ∧ r1==r2)
   →(e1==e2)

Rule 2 states that for all students that take an exam, if the time slot and room are the same, then it must be the same exam.

3) ∀e1, r1, ts1, s1
   (take_exam(e1,s1,r1,ts1) ∧ exam_cap (e1) > room_cap (r1))→
   ¬(take_exam(e1,s1,r1,ts1))

Rule 3 states that for all exams, if a student takes the exam and the exam capacity is greater than the room capacity, then the student cannot take said exam.

4) ∀s1,r1,r2,e1,e2,ts1,ts2
   (e1!=e2 ∧ take_exam(e1,s1,r1,ts1) ∧ take_exam(e2,s1,r2,ts2))
   →(ts1!=ts2)

Rule 4 states that for all students, if they take two different exams, then the time slots cannot be the same.

5) ∀i (0 ≤ i < no_of_slots−1)
   ∀r1,r2,s1,e1,e2
   (¬take_exam(e1,s1,r1,T[i]) V ¬take_exam(e2,s1,r2,T[i+1]))

Rule 5 states that for all students, they may have an exam either in one time slot, or the one right after it, but never both.

```
#Create constants to use for quantifiers
ts1 = z3.Const('ts1', T)
ts2 = z3.Const('ts2', T)
r1 = z3.Const('r1', R)
r2 = z3.Const('r2', R)
s1 = z3.Const('s1', S)
s2 = z3.Const('s2', S)
e1 = z3.Const('e1', E)
e2 = z3.Const('e2', E)
```

Figure 4: Declaring constants for quantifiers

```
#1
solver.add(z3.ForAll([ts1, ts2, r1, r2, s1, s2], z3.Implies(z3.And(e1!=e2,take_exam(e1,s1,r1,ts1),take_exam(e2,s2,r2,ts2)), z3.Or(r1!=r2,ts1!=ts2))))
#2
solver.add(z3.ForAll([ts1, ts2, r1, r2, s1, s2], z3.Implies(z3.And(take_exam(e1,s1,r1,ts1),take_exam(e2,s2,r2,ts2),ts1==ts2,r1==r2),e1==e2)))
#3
solver.add(z3.ForAll([e1, r1, ts1, s1], z3.Implies(z3.And(take_exam(e1, s1, r1, ts1),exam_capacity(e1) > room_capacity(r1)),z3.Not(take_exam(e1, s1, r1, ts1)))))
#4
solver.add(z3.ForAll([s1,r1,r2,e1,e2,ts1,ts2], z3.Implies(z3.And(e1!=e2,take_exam(e1,s1,r1,ts1),take_exam(e2,s1,r2,ts2)), ts1!=ts2)))
#5
for i in range(no_of_slots-1):
  t = T.constructor(i)()
  tPlus1 = T.constructor(i+1)()
  solver.add(z3.ForAll([r1, r2, s1, e1, e2], z3.Or(z3.Not(take_exam(e1, s1, r1, t)), z3.Not(take_exam(e2, s1, r2, tPlus1)))))
```

Figure 5: Rules in z3

Testing

Creating Own Examples

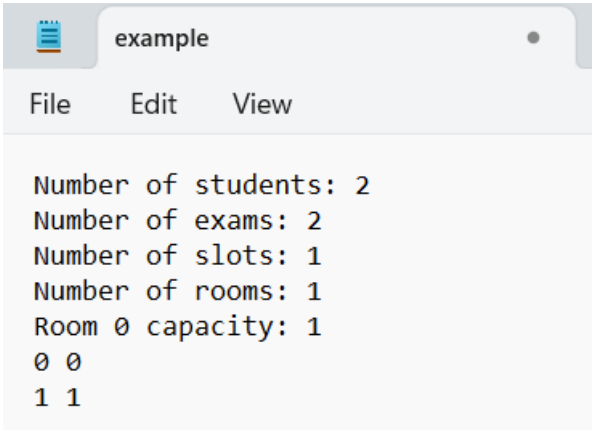
How will we test?

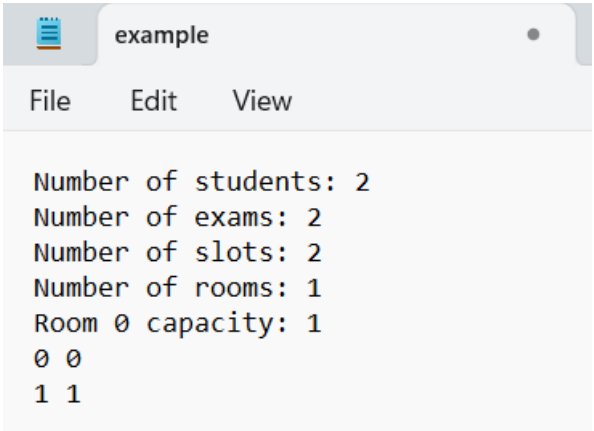
In order to test each rule, we will create an if-else statement that returns “sat” or “unsat” based on the instance. We will use our own example instances which will be shown throughout the report, as it is easier to depict why it works/doesn’t. Figure 6 shows the code used for testing.

```
if solver.check() == unsat:
  print("unsat")
else:
  print("sat")
```

```
test = Instance("example.txt")
solve(test)
```

Figure 6: Code for testing





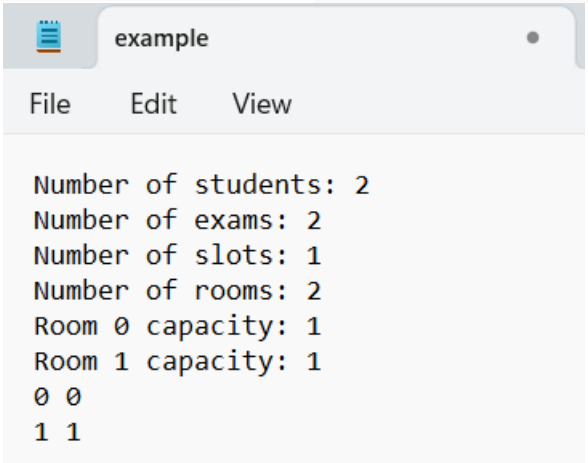


Figure 7: Test instances for rules 1 and 2

Figure 7, is used to test rules 1 and 2 (see page 5), and should print:

- ➔ unsat for top left
  - 1 slot and 1 room but 2 exams, should not work
- ➔ sat for top right
  - 2 exams in 1 room but 2 slots, should work
- ➔ sat for bottom.
  - 2 exams in 1 slot but 2 rooms, should work

My program prints the correct output for each of the examples.

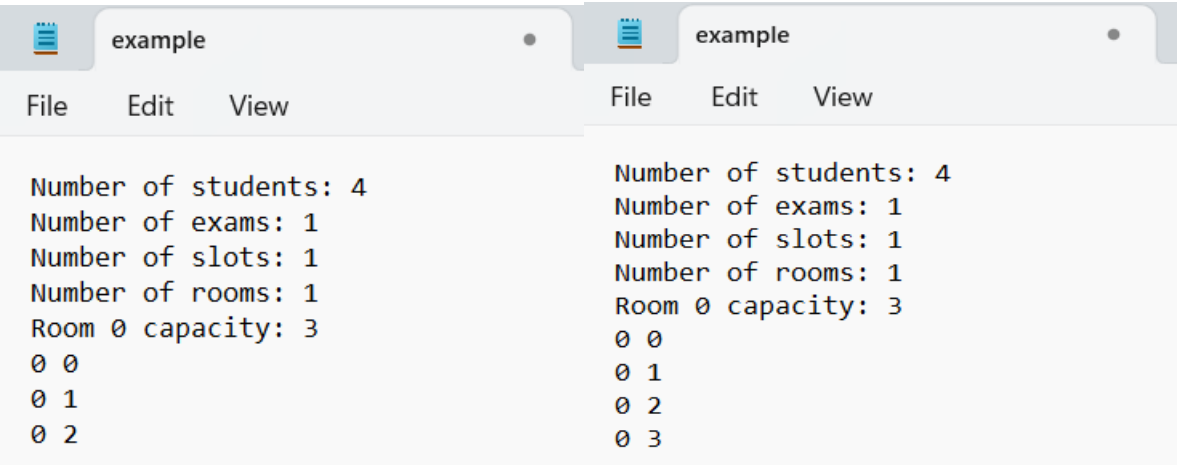


Figure 8: Test instances for rule 3

Figure 8, is used to test rule 3(see page 5), and should print:

- ➔ sat for top left
  - 4 students, 1 exam, 1 room: capacity 3 but only 3 doing exam
- ➔ unsat for top right
  - 4 students, 1 exam, 1 room: capacity 3 and 4 doing exam

My program prints the correct output for each of the examples.

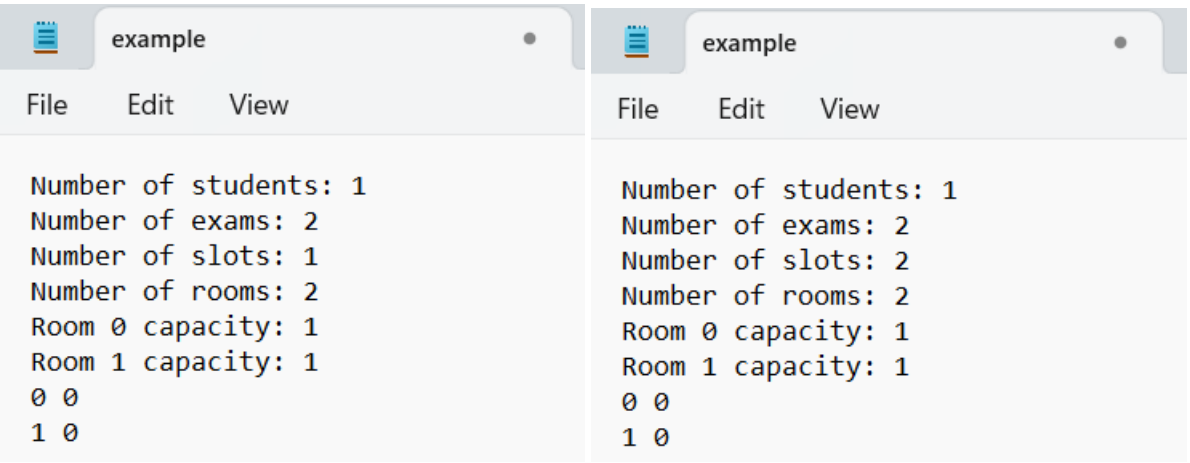


Figure 9: Test instances for rule 4

Figure 9, is used to test rule 4(see page 5), and should print:

- ➔ unsat for top left
  - 1 student, 2 exams, but only 1 time slot
- ➔ sat for top right
  - 1 student, 2 exams, but 2 times slots

My program prints the correct output for each of the examples.

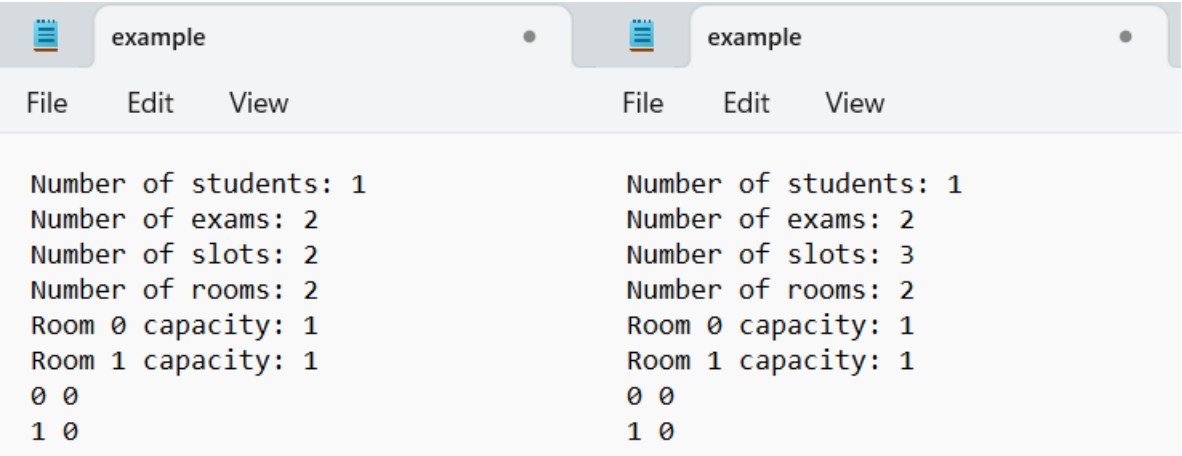


Figure 10: Test instances for rule 5

Figure 10, is used to test rule 5(see page 5), and should print:

- ➔ unsat for top left
  - 1 student, 2 exams, but only 2 time slots, cannot do back to back exams
- ➔ sat for top right
  - 1 student, 2 exams, but 3 times slots, can do time slots 1 and 3

My program prints the correct output for each of the examples.

Testing All Instances

In order to carry out an in-depth test, we create a loop where each instance 1-10 are run. We begin with all the unsat instances, then move on to the sat instances. The output should be 10 unsat, and then 10 sat. Figure 11 shows that the program achieves this.

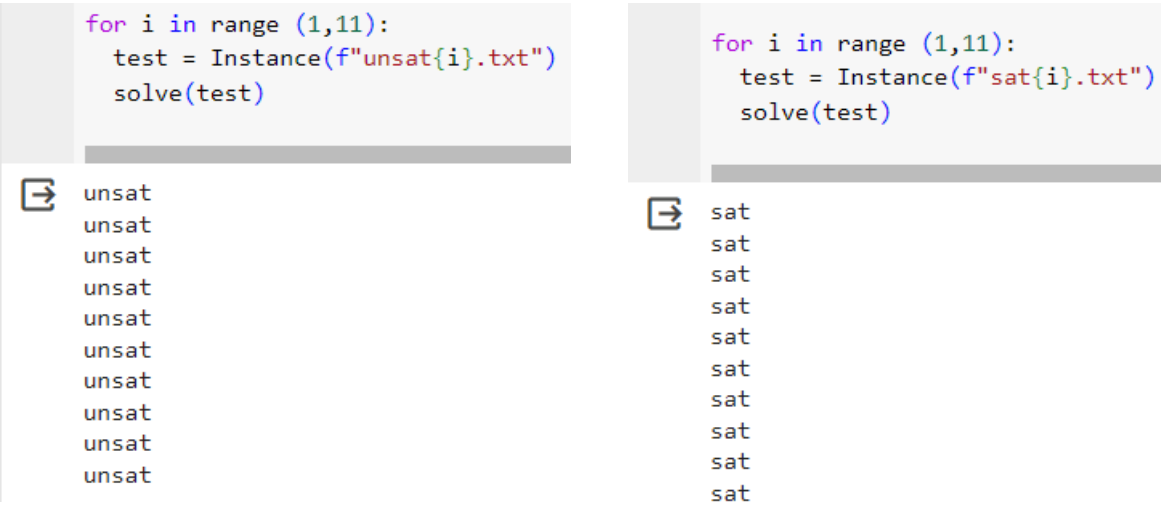


Figure 11: Provided test instances with all 5 rules applied

add_exam() Function

Now, we need to manipulate the solution instance, using predefined 'add_exam()' function, which we cannot do with our 'take_exam()' function as it contains 4 parameters. We will therefore create a new function, called 'addExam()', that takes 3 parameters, same as the function we are given; exam, time slot, room. We will copy the parameters and output from 'take_exam()' and add some rules for the new function.

6) ∀s1, e1, ts1, r1
   (take_exam(e1, s1, r1, ts1) → addExam(e1, r1, ts1))

Rule 6 states that forall students, if they take an exam, then the exam is added to the schedule.

7) ∀r1, r2, ts1, ts2, e1 :
   ((addExam(e1, r1, ts1) ∧ addExam(e1, r2, ts2)) →
   (r1 = r2 ∧ ts1 = ts2))

Rule 7 states that forall exams, if it is the same exam, then the room and time slot are the same.

8) ∀r1, ts1, e1, e2
   ((addExam(e1, r1, ts1) ∧ addExam(e2, r1, ts1)) →
   (e1 = e2))

Rule 8 states that forall exams, if the time slot and room are the same, it is the same exam.

```
#Create new addExam function for add_exam()
addExam = z3.Function('addExam', E, T, R, z3.BoolSort())

#Add constraints
#6
solver.add(z3.ForAll([s1, e1, ts1, r1], z3.Implies(take_exam(e1, s1, r1, ts1), addExam(e1, ts1, r1))))
#7
solver.add(z3.ForAll([r1,r2,ts1,ts2,e1], z3.Implies(z3.And(addExam(e1,ts1,r1),addExam(e1,ts2,r2)), z3.And(r1==r2,ts1==ts2))))
#8
solver.add(z3.ForAll([r1,ts1,e1,e2], z3.Implies(z3.And(addExam(e1,ts1,r1),addExam(e2,ts1,r1)),e1==e2)))
```

Figure 12: New function to store exams, time slots and rooms with rules applied

Printing The Timetable

Now to ensure our timetable works as it is supposed to, we will print timetables for the 10 satisfiable instances using the same method as in Figure 11. 'model.eval' evaluates the parameters within our created 'addExam()' function, and if true it will add the exam to the timetable. Using predefined 'print' function, we will output the timetable.

```
if solver.check() == unsat:
    #If there is no feasible timetable, return None
    return None
else:
    solution = Solution(instance)
    model = solver.model()

    for exam in range(no_of_exams):
        for slot in range(no_of_slots):
            for room in range(no_of_rooms):
                if model.eval(addExam(E.constructor(exam)(), T.constructor(slot)(), R.constructor(room)())):
                    solution.add_exam(exam, slot, room)
solution.print()
```

Figure 13: Code to add exam to timetable and output timetable

```
sat
exam 0: slot 0 room 0
sat
exam 0: slot 2 room 0
exam 1: slot 0 room 0
sat
exam 0: slot 0 room 0
exam 1: slot 1 room 0
sat
exam 0: slot 0 room 1
exam 1: slot 0 room 0
sat
exam 0: slot 0 room 0
exam 1: slot 1 room 0
sat
exam 0: slot 1 room 0
exam 1: slot 3 room 0
exam 2: slot 0 room 0
sat
exam 0: slot 4 room 0
exam 1: slot 2 room 0
exam 2: slot 0 room 3
exam 3: slot 1 room 0
exam 4: slot 0 room 0
```

```
sat
exam 0: slot 4 room 0
exam 1: slot 2 room 0
exam 2: slot 0 room 3
exam 3: slot 1 room 0
exam 4: slot 0 room 0
sat
exam 0: slot 2 room 0
exam 1: slot 4 room 0
exam 2: slot 0 room 0
sat
exam 0: slot 4 room 0
exam 1: slot 6 room 1
exam 2: slot 8 room 1
exam 3: slot 0 room 1
exam 4: slot 2 room 1
sat
exam 0: slot 0 room 1
exam 1: slot 2 room 1
exam 2: slot 9 room 0
exam 3: slot 4 room 0
exam 4: slot 6 room 1
exam 5: slot 0 room 0
```

Figure 14: Example timetable output

Final Test

Finally, as opposed to outputting the timetable, we will run the given 'test()' function, that will use the 10 sat and 10 unsat instances. If the instance is sat, it will return a solution we defined earlier. If it is unsat, it will return none.

```
if solver.check() == unsat:
    #If there is no feasible timetable, return None
    return None
else:
    solution = Solution(instance)
    model = solver.model()

    for exam in range(no_of_exams):
        for slot in range(no_of_slots):
            for room in range(no_of_rooms):
                if model.eval(addExam(E.constructor(exam)(), T.constructor(slot)(), R.constructor(room)())):
                    solution.add_exam(exam, slot, room)
return solution
```

Figure 15: Code to test all instances

```
sat1.txt                    34 ms PASS
unsat1.txt                  37 ms PASS
sat2.txt                    42 ms PASS
unsat2.txt                  31 ms PASS
sat3.txt                    43 ms PASS
unsat3.txt                  60 ms PASS
sat4.txt                    51 ms PASS
unsat4.txt                  58 ms PASS
sat5.txt                    50 ms PASS
unsat5.txt                  62 ms PASS
sat6.txt                    88 ms PASS
unsat6.txt                  50 ms PASS
sat7.txt                   125 ms PASS
unsat7.txt                  49 ms PASS
sat8.txt                   248 ms PASS
unsat8.txt                  57 ms PASS
sat9.txt                   443 ms PASS
unsat9.txt                 436 ms PASS
sat10.txt                  480 ms PASS
unsat10.txt                990 ms PASS
```

Figure 16: Final results

Final Tweaks

Playing around with the code, and commenting out some rules to see how the solver behaves, showed that rules 1 and 2 not needed. This is because in Figure 3, we stated there must be a time slot and room for all student, exam pairs. As we store and manipulate everything in the 'addExam()' function, rules 7 and 8, are essentially rules 1 and 2. They state if 2 exams are the same, that is also the case for their room and time slot. Furthermore, if a room and time slot are the same, it must be the same exam.

```
#3
solver.add(z3.ForAll([e1, r1, ts1, s1], z3.Implies(z3.And(take_exam(e1, s1, r1, ts1),exam_capacity(e1) > room_capacity(r1)),z3.Not(take_exam(e1, s1, r1, ts1)))))
#4
solver.add(z3.ForAll([s1,r1,r2,e1,e2,ts1,ts2], z3.Implies(z3.And(e1!=e2,take_exam(e1,s1,r1,ts1),take_exam(e2,s1,r2,ts2)), ts1!=ts2)))
#5
for i in range(no_of_slots-1):
  t = T.constructor(i)()
  tPlus1 = T.constructor(i+1)()
  solver.add(z3.ForAll([r1, r2, s1, e1, e2], z3.Or(z3.Not(take_exam(e1, s1, r1, t)), z3.Not(take_exam(e2, s1, r2, tPlus1)))))

#Create new addExam function for add_exam()
addExam = z3.Function('addExam', E, T, R, z3.BoolSort())

#Add constraints
#6
solver.add(z3.ForAll([s1, e1, ts1, r1], z3.Implies(take_exam(e1, s1, r1, ts1), addExam(e1, ts1, r1))))
#7 and #1
solver.add(z3.ForAll([r1,r2,ts1,ts2,e1], z3.Implies(z3.And(addExam(e1,ts1,r1),addExam(e1,ts2,r2)), z3.And(r1==r2,ts1==ts2))))
#8 and #2
solver.add(z3.ForAll([r1,ts1,e1,e2], z3.Implies(z3.And(addExam(e1,ts1,r1),addExam(e2,ts1,r1)),e1==e2)))
```

Figure 17: Final implementation of rules

Conclusion

In conclusion, the solver achieves all expected outcomes, and works as intended. The code was minimised to be as efficient as possible, to enhance quality and all the constraints work well, hence the tests being passed.

Additional Features

The main custom feature I will be implementing is an excel spreadsheet, that gives an example timetable for each instance. It will be in the form of Exam, Slot, Room and there will be information each time the timetable represents a new instance in the form 'Instance i', where 'i' represents the number of the instance. Figure 18 describes all of the new code implemented to achieve this functionality, where we import 'workbook', create 'worksheet1' and then append the results of all 10 sat instances to it. They are all then added and separated using a for loop, similar to the one provided by the 'test()' function. Figure 19 displays where to find the created spreadsheet, and Figure 20 shows the output file.

```
for i in range(1, 11):
    # Append a new line for each instance
    worksheet1.append([f'Instance {i}'])
    instance = Instance(f"sat{i}.txt")
    solve(instance)
workbook.save('Time Table.xlsx')
if model.eval(addExam(E.constructor(exam)(), T.constructor(slot)(), R.constructor(room)())):
    solution.add_exam(exam, slot, room)
    worksheet1.append([exam, slot, room])
from openpyxl import Workbook

# Create a new workbook and get the active worksheet
workbook = Workbook()
worksheet1 = workbook.active

# Add headers to the worksheet
worksheet1.append(['Exam', 'Slot', 'Room'])
```

Figure 18: Creating and appending timetable to a spreadsheet

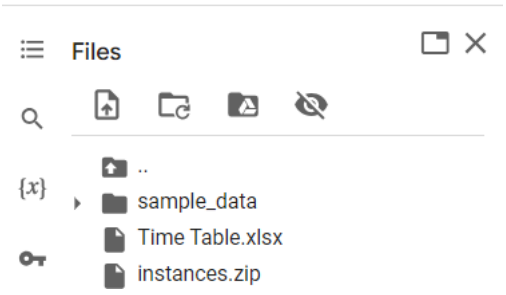
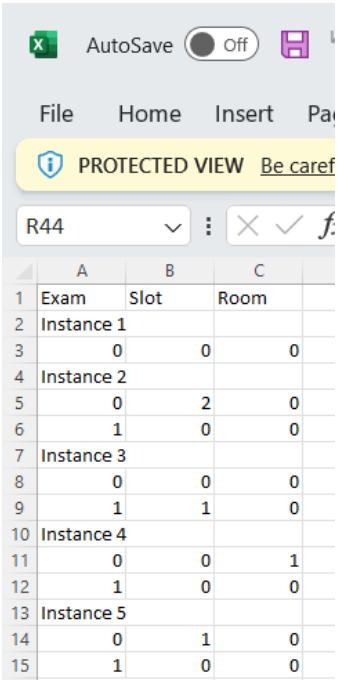


Figure 19: Location of spreadsheet



	A	B	C
1	Exam	Slot	Room
2	Instance 1		
3	0	0	0
4	Instance 2		
5	0	2	0
6	1	0	0
7	Instance 3		
8	0	0	0
9	1	1	0
10	Instance 4		
11	0	0	1
12	1	0	0
13	Instance 5		
14	0	1	0
15	1	0	0
16	Instance 6		
17	0	5	0
18	1	2	0
19	2	0	0
20	Instance 7		
21	0	4	0
22	1	2	0
23	2	0	2
24	3	0	0
25	4	1	0
26	Instance 8		
27	0	4	0
28	1	2	0
29	2	0	0
30	Instance 9		
31	0	0	0
32	1	2	1
33	2	5	1
34	3	9	1
35	4	7	1
36	Instance 10		
37	0	2	1
38	1	6	1
39	2	4	0
40	3	8	0
41	4	0	1
42	5	0	0

Figure 20: Excel spreadsheet output